

Ex 4: Difference between JPA, Hibernate and Spring Data JPA

Java Persistence API (JPA)

- JPA (Java Persistence API) is a Java specification (JSR 338) for object-relational mapping (ORM).
- It defines a standard way to map Java objects to relational database tables.
- JPA itself does not provide an implementation.
- It needs an implementation such as Hibernate to perform database operations.

Hibernate

- Hibernate is an Object Relational Mapping (ORM) framework.
- It is one of the most popular implementations of the JPA specification.
- Hibernate converts Java objects into database records and database records into Java objects.
- It manages sessions, transactions, caching, and automatically generates SQL queries.

Hibernate Example

```
/* to CREATE employee in database */  
  
public Integer addEmployee(Employee employee){  
  
    Session session = factory.openSession();  
  
    Transaction tx = null;  
  
    Integer employeeID = null;  
  
    try{  
  
        tx = session.beginTransaction();  
  
        employeeID = (Integer) session.save(employee);  
  
        tx.commit();  
  
    } catch (HibernateException e) {  
  
        if (tx != null) tx.rollback();  
  
        e.printStackTrace();  
  
    } finally {  
  
        session.close();  
  
    }  
}
```

```
}  
  
    return employeeID;  
  
}
```

Spring Data JPA

- Spring Data JPA is a Spring module built on top of JPA.
- It does not implement JPA itself but uses a JPA provider such as Hibernate.
- It reduces boilerplate code by providing ready-made repository interfaces.
- It automatically provides CRUD operations and integrates transaction management through Spring.

Spring Data JPA Example

EmployeeRepository.java

```
public interface EmployeeRepository extends JpaRepository<Employee, Integer> {  
  
  
  
}
```

EmployeeService.java

```
@Autowired  
  
private EmployeeRepository employeeRepository;  
  
  
  
@Transactional  
  
public void addEmployee(Employee employee) {  
    employeeRepository.save(employee);  
  
}
```